USEREADY / AI ENGINEER CASE STUDY

SolrCloud Observability and Root-Cause Analysis

Complete beginner-friendly technical study guide: flowcharts, coupled synthetic logs, code walkthroughs, worked statistics, LangGraph reasoning, real demo results, evaluation and interview defense.

What this guide covers
The actual V4.1 source code and the successful hosted two-stage Node A investigation. It carefully distinguishes observed evidence, proposed hypotheses and unverified production claims.

WHAT WAS BUILT	DEMONSTRATED RESULT
Deterministic parser	21,609 / 21,609 simulated log lines parsed
Detection and correlation	13 findings grouped into 2 incidents
Adaptive statistics	Node A products p95: 106 ms baseline vs 696 ms observed
Read-only AI diagnostic	504 events examined in the GC/query timeline
Real hosted LLM	Two successful Groq GPT-OSS 120B calls; final JSON validated

Prepared as a learning reference for explaining and defending the working prototype. Source basis: original UseReady assignment PDF, V4.1 project ZIP, exported deterministic report, and saved successful Streamlit investigation.

How to use this guide

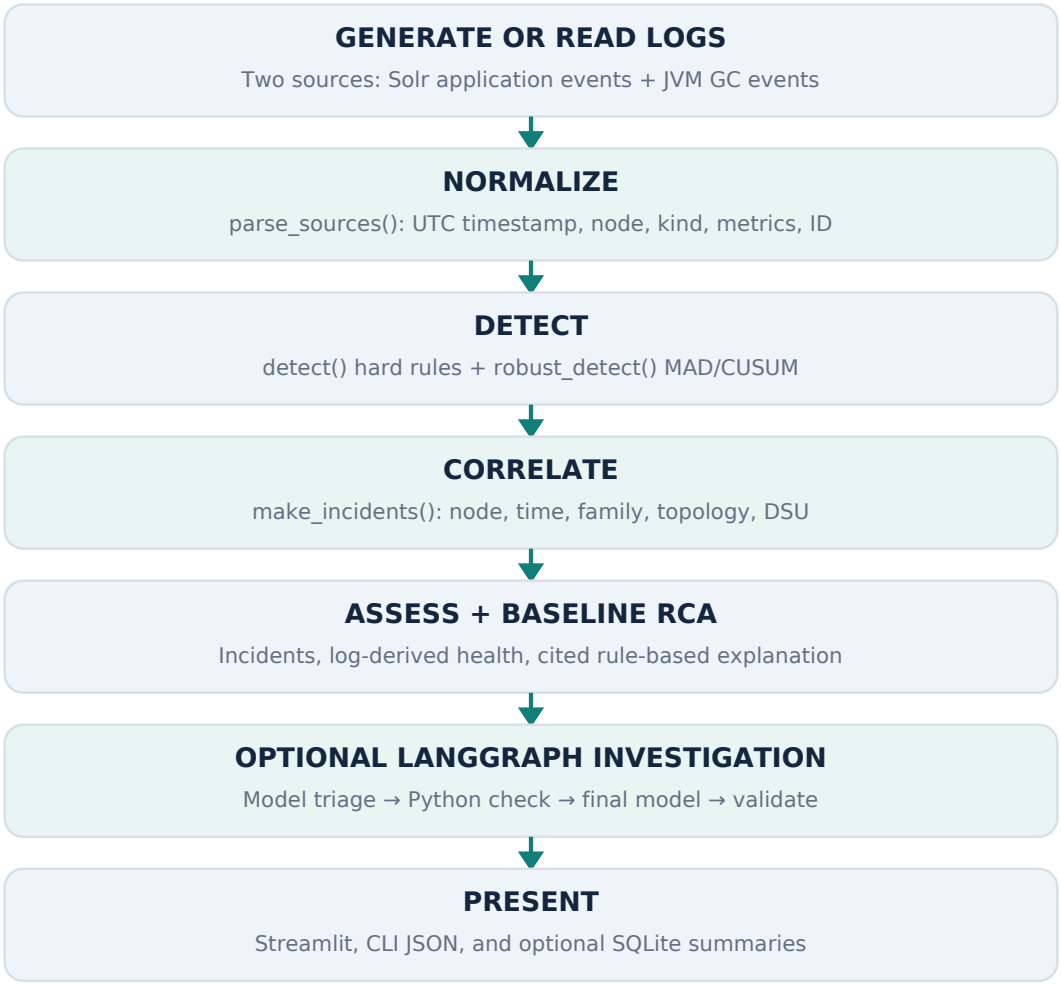
Read Sections 1-4 first to understand the environment and generated data. Sections 5-9 explain the deterministic engine. Sections 10-13 explain LangGraph and the hosted LLM. Sections 14-18 cover validation, tests, deployment boundaries, and how to present the work.

> **The single rule to remember**
> Python extracts measurements, detects symptoms and groups related findings. The LLM interprets a small curated evidence package and requests one allowed diagnostic. A valid model response is still a hypothesis, not proof of causation.

Learning map

CHAPTER	TOPIC / KEY QUESTION
1-2	Problem, SolrCloud/JVM basics and system architecture
3-4	Synthetic generator, coupled Solr/GC logic, parser and event IDs
5-6	Hard thresholds, five-minute windows, median, MAD, CUSUM
7-8	DSU correlation, node/cluster health and deterministic RCA
9-12	Evidence dossier, LangGraph, two-stage Groq call, safety checks
13-15	Actual Node A result, data export, evaluation and rate limits
16-18	Production trade-offs, defense questions, glossary and review

Project at a glance



The arrow from observability to AI is optional. A model failure does not prevent deterministic detection and report generation.

1. The actual business problem

A client uses distributed SolrCloud search on several Linux servers. Engineers troubleshoot incidents by reading large Solr application logs and JVM garbage-collection logs. The assignment asks us to detect operational issues, correlate related events across sources, assess nodes and the cluster, and offer RCA, likely impact and remediation suggestions in a simple interface.

An intuitive example

```
08:39:00  Search completed in 65 ms
08:40:24  Solr request failed: HTTP 503
08:40:40  JVM Full GC pause: 2517 ms
08:42:40  JVM Full GC pause: 2862 ms
08:45:40  JVM Full GC pause: 3004 ms
```

The measurements show degradation, but the first event at 08:40:24 is earlier than the nearby 08:40:40 Full GC event. That is one reason we must not claim those particular failures were caused by that particular pause. The correct conclusion is that the symptoms overlap, and GC may be contributing.

What is SolrCloud?

Solr is a search/indexing platform. A **collection** is a logical searchable dataset, such as `products`. A **shard** is a partition of that collection. A **replica** is a copy of a shard. A **node** is a running Solr server. SolrCloud can distribute shards and replicas across nodes. ZooKeeper traditionally coordinates cluster membership and leader-related state.

TERM	SIMPLE EXPLANATION	EXAMPLE
Collection	Logical searchable dataset	products
Shard	Portion of the collection	products/shard1
Replica	One hosted copy of a shard	Shard 1 on node-a
Node	Solr server / instance	node-b
ZooKeeper	Shared coordination dependency	Node A + B coordination event

What is JVM garbage collection?

Java applications allocate objects in a memory area called the heap. The garbage collector (GC) reclaims memory no longer reachable. Some GC operations pause application progress (stop-the-world). A repeated ~3-second pause is significant for a latency-sensitive search service, but GC timing alone does not identify the initiating cause.

> **What we can claim**
> The detector can report that multi-second pauses, high after-GC occupancy, higher query latency and errors were logged. It cannot infer a memory leak, exact request causality or customer impact without additional evidence.

2. Source files and the main data path

FILE	JOB
generate_logs.py	Create reproducible synthetic Solr + GC records and fault scenarios
parser.py	Parse raw lines into typed events; also contains hard-rule detector
analyzer.py	Adaptive latency detection, incident correlation, health, baseline RCA
agent.py	Evidence selection, API requests, diagnostics, LangGraph and validation
app.py	Streamlit interface; run optional AI investigation
cli.py	Headless deterministic report as JSON
watch.py / storage.py	Local polling and optional SQLite summaries
evaluate.py / tests/	Scenario comparisons and regression tests

```
# Main deterministic path in analyzer.py
events, quality = parse_sources(sources)
raw = baseline_detect(events)
rows = derived_features(events)
learned, stats = robust_detect(rows, events)
findings = normalize_findings(raw + learned)
incidents, edges = make_incidents(findings, events, topology)
```

The variable `learned` in that code is only a local name: the MAD/CUSUM detector is statistical, not a trained ML model. The Streamlit V4 app uses `analyzer.analyze()`, which imports the original hard-rule `parser.detect()` as `baseline_detect()`. `parser.py` also contains an older standalone analysis path.

3. Synthetic log generation: how it works

We had no authentic client logs to run locally. The generator produces realistic-looking but simulated records. It uses a seeded pseudorandom number generator, minute-by-minute request traffic, a per-node heap state and deliberately injected failures. It does not start a real Solr, Java or ZooKeeper process.

Inputs and reproducibility

```
def gen(scenario='mixed', seed=23,
        duration_minutes=105, rate=64,
        start='2026-10-26T08:00:00+00:00'):
    rnd = random.Random(seed)
    heap = {f'node-{n}': 320.0 + rnd.random()*60 for n in 'abc'}
    # Simulate workload minute by minute
```

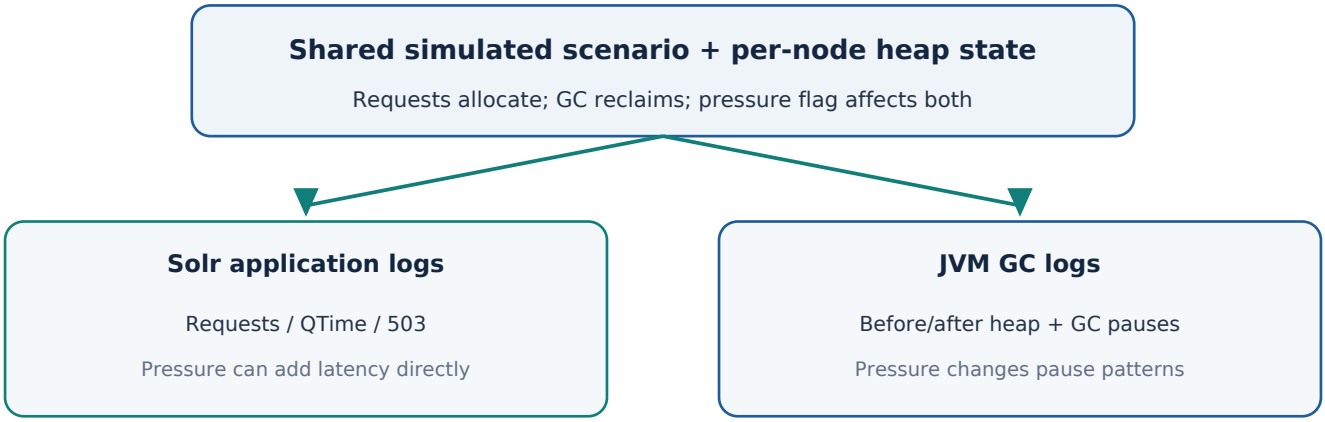
`scenario` chooses which fault appears. `seed` makes runs reproducible. `rate` is the approximate baseline requests per minute per node; actual counts vary. `duration_minutes` sets the time span; the bundled run is 105 minutes. `start` sets the simulated UTC timestamp.

Why query latencies look variable

```
seasonal = 1 + .40 * math.sin(2*math.pi*minute/54)
surge = 1.65 if 22 <= minute < 27 else 1.0
n = max(8, round(rate*seasonal*surge*rnd.uniform(.85,1.15)))
complexity = 1.8 if 'facet=true' in query else 1.0
base = rnd.lognormvariate(math.log(38), .42) * complexity
base *= (1 + max(0, n-rate)/rate*.50)
```

Traffic follows a smooth wave, a short surge and random jitter. Request times are sampled from a lognormal distribution, producing many ordinary requests with a longer right tail. Faceted searches receive a complexity multiplier; higher-than-baseline traffic also increases simulated latency.

How the two generated log streams are connected



Important: no real JVM is simulated

Query durations are not derived from individual stop-the-world GC intervals

They are separate files, but not independently generated random streams. In the generator, each request raises `heap[nod]`. The GC path later reads that heap state, emits before/after values, and writes its reclaimed value back into the same heap state. The `pressure` flag additionally changes both request-time behavior and GC frequency/duration.

```
# Solr-side allocation (per simulated request)
heap[nod] += (rnd.uniform(.12,.42) * complexity
              * (3.0 if pressure else 1.0))

# Query latency also gets a direct stress boost
if pressure and coll == 'products':
    base += rnd.uniform(320,660)

# GC reads and updates the same per-node heap
before = min(1020, int(heap[nod] + rnd.uniform(60,95)))
# ... choose after and pause based on pressure ...
heap[nod] = float(after)
```

The distinction matters. There is **stateful coupling of requests, allocations and GC**, and there is also a **common injected stress condition** affecting both streams. But the simulator does NOT halt query threads during a GC pause and compute the resulting exact QTime. Therefore the observed co-movement is by construction, not proof that GC pauses caused individual 503s.

> **How to say this in the interview**
> "Solr and GC events are emitted into separate files, but their generation is coupled by a shared per-node heap state and fault flag. This produces coherent fault scenarios for testing correlation. It does not model the real JVM scheduling and pause/request causal mechanics."

Fault scenarios and artifacts

SCENARIO	WHAT IS INJECTED / WHY IT IS USEFUL
normal	Only ordinary activity; negative control
gc	Node A allocation/GC pressure plus query degradation
zk	Shared ZooKeeper session/leader events on A and B
slow_no_gc	Slow product queries without GC pressure
benign_zk	Brief disconnect and reconnect; should not escalate
oom	Explicit OutOfMemoryError on node-c
independent	GC issue on A and independent ZK issue on B
mixed	Node A memory/query incident plus later A+B ZK incident

The output is six `.log` files plus `TOPOLOGY.json` and `GROUND_TRUTH.json`. Topology is used for correlation. Ground truth records the injected scenarios for evaluation and is never fed to the detector or LLM.

Commands to generate your own logs

```
# From the project directory in PowerShell
.\.venv\Scripts\python.exe generate_logs.py --scenario mixed --seed 42 --output-dir
generated_logs\my_test
.\.venv\Scripts\python.exe generate_logs.py --scenario slow_no_gc --seed 77 --output-dir
generated_logs\slow_test
.\.venv\Scripts\python.exe generate_logs.py --scenario gc --seed 55 --duration-minutes 120 --rate
100 --output-dir generated_logs\high_load
```

Then run `cli.py --dir generated_logs\my_test --nodes node-a,node-b,node-c --output my_report.json`, or point Streamlit at the generated folder. Authentic external logs are supported only where their formats match the parser; adaptation and data sanitization may be necessary.

4. Parsing: text becomes structured events

The parser turns loosely formatted strings into consistent Python dictionaries, letting later code use numbers and timestamps rather than repeatedly interpret log text.

One GC record, step by step

```
[2026-10-26T08:45:40.000+0000][info][gc]
GC(53) Pause Full (G1 Compaction Pause)
1020M->998M(1024M) 3003.7ms
```

The record says that heap occupancy fell from 1020 MB to 998 MB out of a 1024 MB reported capacity, and the Full GC operation paused for 3003.7 ms. The parser normalizes its timestamp and event type and extracts the numbers.

```
{
  "node": "node-a", "source": "gc",
  "kind": "full_gc",
  "measures": {
    "before_mb": 1020, "after_mb": 998,
    "heap_mb": 1024, "pause_ms": 3003.7,
    "heap_after_pct": 97.5
  }, "filename": "node-a_gc.log", "line": 56
}
```

Compute after-GC occupancy: `998 / 1024 * 100 = 97.46%`, rounded to 97.5%. GC reclaimed only 22 MB from a heavily occupied heap. That is evidence of high retained occupancy, but it does not identify a memory leak.

Parsing Solr requests

```
# Representative synthetic request line
2026-10-26 08:40:24.236 ERROR SolrCore.Request
[collection=products shard=shard1 replica=node-a]
path=/select params={q=headphones&rows=20}
QTime=387 status=503 http_status=503 request failed
```

The parser classifies this as `http_5xx`, extracts `qtime_ms=387`, and preserves collection, shard, replica and trace IDs when present. Solr QTime is not necessarily end-to-end user latency.

Regular expressions and timestamps

```
QTIME = re.compile(
    r"\b(?:QTime|qtime|elapsed(?:Time)?)\s*[=:]"
    r"\s*(\d+(?:\.\d+)?)\s*(?:ms)?\b", re.I)
# parse_timestamp(): ISO-like timestamp -> aware UTC datetime
```

A regex matches patterns rather than specific literal strings. The captured number after `QTime=` becomes a float. The time parser accepts several ISO-like forms and converts offsets to UTC; if a timestamp has no timezone, it assumes UTC. Confirm actual timezone settings for any real deployment.

Event IDs, duplicates and multiline traces

```
fingerprint = hashlib.sha256(
    f"{node}|{source}|{ts.isoformat()}|{line}".encode("utf8")
).hexdigest()[:14]
```

The 14-character hash is a convenient deterministic evidence reference. A separate `filename:line` pointer enables human inspection. These references do not prove authenticity. `parse_sources()` also tracks duplicate fingerprints, counts unparsed lines, associates recognized stack-trace continuations with the preceding event, and sorts by UTC timestamp.

> **Why IDs matter to the LLM**
> A model may propose a cause, but it must cite actual IDs from the dossier. Python rejects references that do not exist. That protects against invented citations, though it does not establish that the cited event semantically supports the claim.

5. Deterministic detection: hard rules

A **raw event** is a recorded log message. A **finding** is a judgment produced by a documented rule. The hard-rule detector in `parser.detect()` first groups events by `(node, five-minute bucket)`.

```
def bucket(ts, seconds=300):
    return int(datetime.fromisoformat(ts).timestamp()) // seconds

for event in events:
    groups[(event["node"], bucket(event["ts"]))].append(event)
```

A tumbling window has fixed boundaries, such as 08:40:00-08:44:59 UTC. This simplifies counting, but incidents straddling boundaries can appear in adjacent buckets.

SIGNAL	TRIGGER	SEVERITY
Long GC pause	At least one pause >= 500 ms	Warning; critical at >= 2000 ms
Post-GC heap	After-GC occupancy >= 85%	Warning
Frequent Full GC	At least 2 Full GC events per 5 min	Warning
OOM	Explicit OutOfMemoryError indicator	Critical
ZooKeeper	Logged disconnection / session trouble	Critical (subject to transient filter)
Leader churn	At least 2 election/change events per 5 min	Warning
Request errors	At least 2 error/5xx per 5 min	Warning; critical at >= 4
Slow queries	At least 2 QTime values >= 1000 ms	Warning

Worked rule 1: long GC

A window contains GC pauses of 40, 60, 85, 2517 and 2862 ms. The detector filters records with `pause_ms >= 500` and finds the worst. Because 2862 >= 2000, it creates a critical `gc_pause` finding with supporting event IDs.

```
pauses = [e for e in gc if e["measures"].get("pause_ms",0)>=500]
if pauses:
    worst = max(pauses, key=lambda e:e["measures"]["pause_ms"])
    severity = "critical" if worst["measures"]["pause_ms"]>=2000 else "warning"
```

Worked rule 2: high retained heap

After the example Full GC, 998/1024 = 97.5% of heap remains occupied. Because 97.5 >= 85, a separate warning-level `heap_pressure` finding appears. Why separate it from GC pause? Duration and retained heap describe different failure symptoms.

Worked rule 3: HTTP error burst

Five logged HTTP 503/error entries in one five-minute bucket exceed the critical count of four. This is an **error count among observed logs**, not an authoritative traffic-normalized error rate; logs may not contain every incoming request.

Transient ZooKeeper events

The upgraded `analyzer.analyze()` suppresses a narrow case: one non-expired ZooKeeper disconnection followed by an explicit same-node reconnect within 30 seconds. It does not suppress an expired session or longer disruption. This is exercised by the `benign_zk` negative-control scenario.

> **Main hard-rule limitation**
> Thresholds are initial heuristics. A global 1000 ms slow-query rule can miss a jump from 50 to 500 ms even though the workload became ten times slower. That motivated the adaptive statistical detector.

6. Adaptive latency: p95, median, MAD and CUSUM

Step 1: build minute-level features

`derived_features(events)` groups Solr events by `(node, collection, UTC minute)` and computes request count, logged 5xx count, logged error fraction, and p95 query-processing latency.

```
key = (node, collection, minute)
values = sorted(e["measures"]["qtime_ms"] for e in group)
position = max(0, math.ceil(.95*len(values))-1)
p95_qtime_ms = values[position]
```

p95 is the 95th percentile: under the nearest-rank method, approximately 95% of measured values are at or below it. For the sorted values `[30,35,40,42,45,50,55,60,70,500]`, the nearest-rank p95 is 500 ms. Small minute samples produce noisier tail estimates.

Step 2: baseline median

`robust_detect()` requires at least 12 previous eligible minute-level p95 values for the same `(node, collection)`. It keeps a rolling baseline of up to 30 values. It excludes minutes with fewer than 8 logged requests from the evaluation.

Example baseline: `[90, 95, 100, 105, 110]` ms. Its median is 100 ms. We use five numbers here only to make the calculation easy; the implemented detector waits for at least 12 historical eligible minutes.

Step 3: median absolute deviation (MAD)

Compute absolute distances from median 100: `[10,5,0,5,10]`. The median of these distances is 5 ms. MAD expresses typical variability without being overly sensitive to rare spikes.

```
med = statistics.median(baseline)
mad = statistics.median(abs(value-med) for value in baseline)
scale = max(10.0, 1.4826*mad)
robust_z = (current_p95-med)/scale
```

The factor 1.4826 makes MAD comparable to standard deviation for normally distributed values; the 10 ms floor prevents a near-zero denominator. For `med=100`, `MAD=5`, `x=500`, scale=max(10,7.413)=10 and robust Z=(500-100)/10=40.

Step 4: require an operationally meaningful increase

```
surprising = robust_z > 6 and x > max(med*2.5, med+100)
```

A large Z-score alone is insufficient. The current p95 must also be more than 2.5 times the median and over 100 ms above it (because it must exceed the maximum of those two thresholds). This guards against alerts for tiny absolute deviations on very-fast workloads.

Step 5: positive CUSUM

CUSUM accumulates evidence of a sustained upward change. At each minute:

```
allowance = max(20, 0.6*med)
cusum = max(0, cusum + x-med-allowance)
shifted = cusum > max(350, 5*med)
```

Worked example: median 100 ms, current p95 stays at 500 ms, so allowance=max(20,60)=60. Each elevated minute adds `500-100-60=340`. Starting from zero, CUSUM becomes 340, 680, 1020, 1360. The shift threshold is max(350,500)=500, so minutes 2-4 pass `shifted`.

Step 6: persistence

```
if surprising and shifted:
    run.append(row)
else:
    run = []
    cusum = max(0.0, cusum*.25)

if len(run) == 3 and not detected:
    # emit one latency_shift finding
```

Three **consecutive qualifying minutes** are required. In our 500 ms toy example, the first minute is surprising but not yet CUSUM-shifted. The next three qualify, so the finding fires on the fourth elevated minute. The detector temporarily freezes baseline updates around an ongoing detection to avoid quickly learning an outage as normal.

Actual Node A adaptive detection

MEASUREMENT	VALUE
Collection / node	products / node-a
Historical minute-p95 median	106 ms
Observed minute-p95	696 ms
Ratio to historical baseline	about 6.6x
Robust Z-score	24.1
Detection time	08:42 UTC, 26 October 2026

A fixed 1000 ms slow-query threshold could miss this 696 ms p95 shift. MAD/CUSUM sees it as a large, persistent deviation relative to normal behavior.

> **MAD versus CUSUM**
> MAD/robust Z asks: “Is this minute unusually slow relative to the recent baseline?” CUSUM asks: “Have upward deviations accumulated?” Persistence asks: “Did enough consecutive minutes qualify?” They address different problems.

7. Correlation and Disjoint Set Union (DSU)

The detector produced **findings**, not root causes. We must decide which findings belong in the same operational incident. `make_incidents()` pairs findings only when timing, node identity and domain relationships permit a link. It then uses DSU/Union-Find to group connected components.

FAMILY	FINDING TYPES
Memory	gc_pause, frequent_full_gc, heap_pressure, oom
Query	slow_queries, latency_shift, error_burst, error_rate_shift
Coordination	zk_disconnect, leader_churn

The actual correlation rules

- On the **same node**, findings link if their first-seen timestamps are no more than 6 minutes apart and they belong to the same family, or one is memory and the other query.
- **Across nodes**, findings link only if both are coordination-family events, they are within 3 minutes, and both nodes are named in the supplied ZooKeeper dependency list.
- Memory and coordination events do not merge merely because they share a node and time.
- Every accepted edge is recorded with a descriptive relationship and `causal: false`.

```
if gap > 6:
    continue
if a["node"] == b["node"]:
    if fa == fb or {fa,fb} == {"memory","query"}:
        relation = "same_node_temporal" if fa==fb else "possible_gc_query_relation"
    else:
        continue
elif (fa == fb == "coordination"
      and a["node"] in zk_nodes and b["node"] in zk_nodes
      and gap <= 3):
    relation = "shared_zookeeper_dependency"
else:
    continue
join(i,j)
```

Why DSU?

Suppose GC pause (A) links to heap pressure (B), heap pressure links to query slowdown (C), and slowdown links to HTTP errors (D). A, B, C and D form a connected component. Union-Find provides a simple way to discover that component.

```
parent = list(range(len(findings)))
def root(i):
    while parent[i] != i:
        parent[i] = parent[parent[i]]  # path compression
        i = parent[i]
    return i
def join(a,b):
    parent[root(a)] = root(b)
```

Each finding initially belongs to its own group. `root()` finds a representative; `join()` merges connected groups. Path compression shortens parent chains. The implementation does not add union-by-rank, which could further optimize pathological cases.

Why timing alone is insufficient

A JVM GC pause and a ZooKeeper disconnect might happen near each other by coincidence. Correlating every simultaneous log message would over-merge incidents. Our family-based and topology-based checks deliberately restrict permissible links.

> **DSU trade-off**
> Connectedness is transitive: if A links to B and B links to C, A and C end up grouped even without a direct relationship. This can help represent an evolving incident, but it can also over-merge. The edge rules are therefore crucial.

8. Incidents, health and deterministic fallback

What our bundled demo actually produced

MEASURE	VALUE
Input records	21,609 synthetic log lines
Parsed events	21,609; no unparsed lines in generated input
Findings	13
Incidents	2
Incident 1	b2293e1c99: node-a memory/query degradation
Incident 2	acb4f14254: node-a + node-b ZooKeeper coordination
Node health	A CRITICAL; B CRITICAL; C NO_ANOMALY_OBSERVED
Cluster assessment	CRITICAL (multi-node critical incident)

Severity, node health and cluster health

An incident is `critical` if any member finding is critical; otherwise warning. A node is CRITICAL if involved in a critical incident, DEGRADED for noncritical incidents, NO_ANOMALY_OBSERVED if it has parsed data but no incident, and UNKNOWN if expected but no events are observed.

Cluster condition is CRITICAL for a multi-node critical incident; otherwise it becomes DEGRADED if any incident exists. If no incidents exist but expected node data is absent, it may be UNKNOWN. **These are log-derived assessments**, not a live Solr cluster-state check.

Baseline RCA uses rules before the LLM

```
if "oom" in types:
    diagnosis = "JVM OutOfMemoryError is logged; initiating cause unknown."
elif "zk_disconnect" in types:
    diagnosis = "Solr node(s) reported ZooKeeper session trouble."
elif memory_issue and query_issue:
    diagnosis = ("Memory/GC pressure and degraded queries overlap; "
                 "contribution plausible.")
else:
    diagnosis = "Operational anomaly detected; unique cause unverified."
```

Illustrative excerpt condensed from the decision branches in make_incidents().

For Node A the saved exact diagnosis was: “JVM memory/GC pressure and degraded queries overlap; GC contribution is plausible but not established.” That conclusion exists even without an API key or LangGraph. It is the fallback when the optional model cannot provide an acceptable answer.

9. Preparing the LLM evidence dossier

The LLM does not receive all 21,609 events. `prepare_evidence()` looks up incident evidence IDs and selects representative observations, preferring large GC pauses, high retained heap and relevant error/query samples. It also tries to cover each involved node.

```
by_id = {event["id"]: event for event in events}
# For each finding: choose a relevant representative event.
# Include source filename/line, timestamp, metrics and bounded excerpt.
# Add a few aggregate statistics, topology and runbook checks.
```

The dossier contains incident ID, nodes, anomaly types, an initial deterministic hypothesis, detector confidence, original observations with exact IDs, read-only summary statistics, selected topology, runbook checks and explicit caveats.

Exact evidence preview from the successful Node A run

DOSSIER FIELD	RECORDED VALUE
Original log excerpts	6
Approximate Stage 1 input estimate	1,472 tokens
Nearby GC pauses	18
Worst GC pause	3,003.7 ms
Logged queries	479
Largest observed QTime	741 ms
Logged HTTP 5xx	32
Detector confidence	medium

Those counts are computed by Python over a bounded nearby scope, not invented by the LLM. The initial deterministic hypothesis is included in the dossier: this can help the model, but may also anchor it toward the baseline diagnosis.

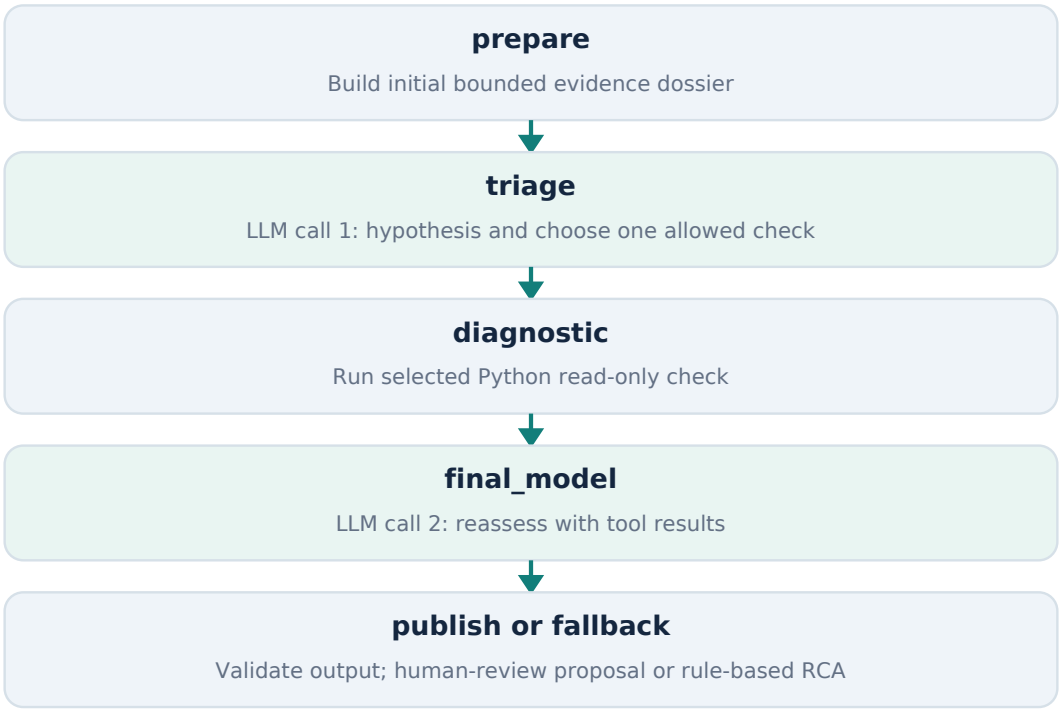
Input-token budgeting

```
estimated = (len(prompt.encode("utf-8"))+2)//3 + 80
usable = min(input_budget_tokens,
             context_window_tokens-max_output_tokens-256)
# Trim selected observations until estimated prompt fits.
```

This is a **rough estimate**, not Groq's exact tokenizer. It reduces accidental oversized requests but cannot guarantee an exact token ceiling. In the successful run the Stage 2 prompt used 2,613 provider-counted input tokens even though the UI planning budget was 2,560.

10. LangGraph: a workflow, not a model

LangGraph provides shared state, nodes, edges and conditional routing. Groq runs the actual reasoning model. Our graph is a *single bounded investigation workflow*, not a fleet of autonomous agents.



The real graph also contains conditional retries from `triage` and `final_model`, and branches to `fallback` when attempts are exhausted. A diagnostic failure also falls back.

State fields

FIELD	WHAT IT STORES
incident / events	Selected incident and parsed local event data
evidence / final_evidence	Bounded first and second model dossiers
initial / diagnostic	Stage 1 JSON and permitted tool result
draft / result	Candidate final JSON and published response
error / attempts / accepted	Validation and retry-control state
steps / total_usage	Execution trace and provider token counters

```
graph = StateGraph(InvestigationState)
graph.add_node("prepare", prepare)
graph.add_node("triage", triage)
graph.add_node("diagnostic", diagnostic)
graph.add_node("final_model", final_model)
graph.add_node("publish", publish)
graph.add_node("fallback", safe_fallback)
graph.add_edge(START, "prepare")
graph.add_edge("prepare", "triage")
# Conditional edges choose retry, next stage, or fallback.
return graph.compile()
```

Plain Python functions and `if` statements could implement this small workflow. LangGraph makes the state transitions and failure routes explicit and supplies a path to checkpointing later. The present graph does not have a durable checkpointer configured.

11. Stage 1: the model chooses a diagnostic

`TRIAGE_SYSTEM_PROMPT` asks the model for exactly four primary fields: preliminary hypothesis, cited existing evidence IDs, one approved diagnostic name or `none`, and an explanation of why the check matters.

```
{
  "preliminary_hypothesis": "GC pressure may contribute to failures",
  "evidence_ids": ["d15ea69ba6211c", "234c6bbf255178"],
  "requested_check": "gc_query_timeline",
  "why_this_check": "Compare minute-aligned GC and query behavior"
}
```

Shortened teaching example; the saved actual text was longer.

Safety: the model chooses a name, not arbitrary code

ALLOWLISTED NAME	WHAT THE IMPLEMENTED PYTHON FUNCTION DOES
gc_query_timeline	Minute GC counts, max pause, logged query p95, HTTP 5xx
indexing_activity	Counts indexing-related log messages, event types
zookeeper_timeline	Summarizes observed session/leader events
error_breakdown	Logged errors, types and query p95
none	No extra diagnostic

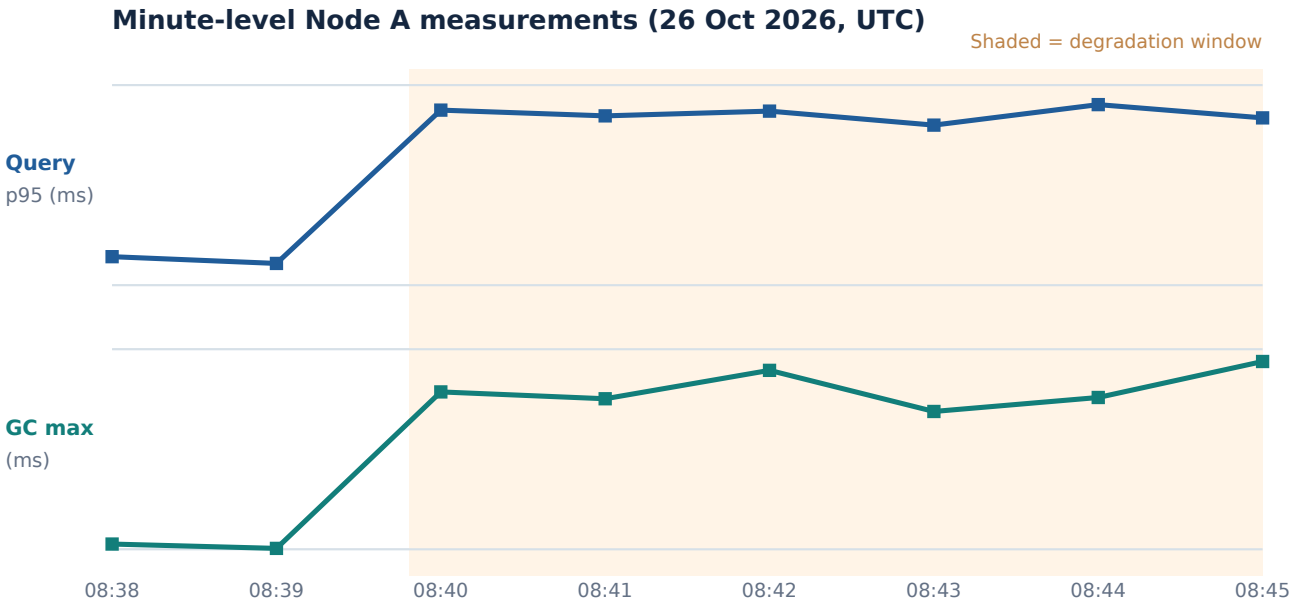
`check_triage()` rejects invented evidence IDs and unapproved tool names. The model cannot request arbitrary shell commands, browse arbitrary files, restart Solr or modify the cluster. `run_read_only_check()` only examines already-parsed events in memory, using constrained time windows and a maximum number of excerpts.

What really happened in the Node A run

The first GPT-OSS 120B call proposed: “Frequent long Full GC pauses are coinciding with spikes in 5xx errors and high query latency, suggesting GC pressure may be contributing to request failures.” It chose `gc_query_timeline` to compare GC and query timing. Stage 1 validation passed.

12. Python diagnostic and Stage 2 final RCA

The diagnostic was run over Node A events within six minutes on each side of the incident anchor. It examined 504 events and produced per-minute GC and request summaries. Only the bounded summary and a few extra log excerpts were passed to Stage 2.



Two mini-plots use different y-axis scales: query p95 is shown on top, maximum GC pause below. The orange area marks the simulated degradation window. Both change abruptly around 08:40 UTC.

UTC MIN	MAX GC PAUSE	QUERY P95	HTTP 5xx
08:38	84.1 ms	114 ms	0
08:39	14.7 ms	87 ms	0
08:40	2517.2 ms	700 ms	3
08:41	2407.8 ms	677 ms	5
08:42	2862.1 ms	696 ms	4
08:43	2204.4 ms	640 ms	8
08:44	2428.1 ms	722 ms	5
08:45	3003.7 ms	669 ms	7

Why this helps

Before 08:40, the observed minute-level query p95 was about 82-114 ms and maximum GC pauses were generally short. Starting at 08:40, query p95 moved into the 640-722 ms range while maximum pause durations were 2.2-3.0 seconds and 5xx errors appeared. This supports temporal association, not proof of direct request-level causality.

Stage 2 input and output

`prepare_final_dossier()` combines initial evidence, the Stage 1 assessment and the Python diagnostic. `FINAL_SYSTEM_PROMPT` asks for 1-3 hypotheses, reasons, real evidence IDs, qualitative confidence, potential impact, manual checks and limitations.

```
{
  "hypotheses": [
    {"cause": "Potential GC-related request stalls",
     "reason": "Long GC pauses overlap elevated query p95 and errors",
     "evidence_ids": ["b90ef80bf89884", "234c6bbf255178"],
     "confidence": "medium"}
  ],
  "potential_impact": "Search responsiveness may be degraded",
  "manual_checks": ["Align actual GC and request start/end intervals"],
  "limitations": "Log correlation does not establish causality"
}
```

Teaching example preserving the implemented output schema.

In the saved run, the model produced **two** hypotheses: (1) long Full GC pauses possibly contributing to request stalls (medium confidence), and (2) workload-driven memory pressure as an alternative (low confidence). It proposed manual investigation of heap, caches, allocation patterns, indexing and query parameters.

13. Validation and model-safety boundaries

A fluent LLM explanation is not inherently correct. We therefore parse its JSON, check the required shape and permitted values, validate evidence IDs, and constrain some unsafe overclaims.

What `check_response()` actually checks

- Output must be a JSON object with 1-3 hypothesis objects.
- Each hypothesis needs meaningful `cause` and `reason`, a nonempty list of known evidence IDs, and `low` / `medium` / `high` confidence.
- Confidence may not exceed the deterministic detector's heuristic cap; `apply_confidence_policy()` normalizes and logs adjustments.
- Impact and limitations text must exist; manual checks must be a list of 1-6 meaningful strings.
- Some definitive-causality phrases and fabricated quantitative business impacts are rejected.
- If invalid after allowed retries or if the API fails, the graph returns `RULE_BASED_FALLBACK`.

An example citation check

```
allowed = {e["id"] for e in evidence["observations"]}
if any(eid not in allowed for eid in hypothesis["evidence_ids"]):
    return "A hypothesis cited a missing or invented evidence ID"
```

This proves that the cited IDs were included in the dossier. It does **not** prove that their contents support the exact statement being made. JSON mode also does not enforce our whole schema, so Python validation remains necessary.

> **A real limitation we discovered**
> The accepted answer said “Long Full GC pauses causing request thread stalls leading to 5xx errors”. This phrasing overstates what minute-level co-occurrence proves. Our pattern-based validator did not catch that formulation. The safer explanation is “GC pauses may contribute; request-level causation is unverified.”

Other boundaries

Raw log excerpts are marked untrusted in the prompt. Tool choice is allowlisted. The model has no automatic remediation permission and cannot change Python-generated severity. However, prompt-injection resistance is not absolute, and customer log data would require privacy controls and redaction before any hosted API call.

14. Hosted inference, quotas and actual model usage

On your Windows machine, Streamlit, parsing, LangGraph and Python diagnostics run locally. The GPT-OSS 120B inference runs on Groq servers through an OpenAI-compatible HTTPS endpoint. The user enters an API key in the UI instead of hardcoding it in source.

```
base_url = "https://api.groq.com/openai/v1"
model = "openai/gpt-oss-120b"
request_data = {
   "model": model,
   "messages": [
      {"role":"system", "content": system_prompt},
      {"role":"user", "content": json.dumps(dossier)}
   ],
   "max_completion_tokens": 1536,
   "response_format": {"type":"json_object"},
   "reasoning_effort": "low"
}
# urllib.request POST to base_url + '/chat/completions'
```

Our successful configuration used temperature 0, low reasoning effort, six initial excerpts, approximate input budget 2,560 tokens, output ceiling 1,536 tokens per call and two additional diagnostic excerpts.

Real recorded result

METRIC	TRIAGE (STAGE 1)	FINAL (STAGE 2)
Prompt tokens	1,529	2,613
Completion tokens	210	535
Total tokens	1,739	3,148
Provider reported total time	~0.51 s	~1.23 s

The two completed model calls used 4,887 total tokens. The reported provider times exclude parts of the full user-facing workflow, such as local processing or interface overhead. We successfully observed `AI_PROPOSAL_REQUIRES_HUMAN_REVIEW`.

What went wrong during development?

We first encountered HTTP 403. A separate GET `/models` check showed the key could list models, but that alone did not prove POST generation access. The corrected HTTP client sent consistent headers, stripped whitespace around the key and exposed the provider's actual error response, after which generation worked.

Next we encountered HTTP 429: Groq reported an 8,000-token-per-minute quota and a final-stage request that would exceed the remaining allowance. An immediate same-size retry also failed. We reduced evidence/output budgets and later obtained a successful two-stage result. A production client should respect provider backoff guidance instead of immediately retrying every 429.

> **Never overstate LLM selection**
> GPT-OSS 120B is a demonstrated working baseline. We have not shown it is the most accurate or cheapest model for Solr RCA, nor that two stages outperform one. Those claims would require a controlled benchmark.

15. Streamlit, CLI, watch mode and data persistence

INTERFACE	WHAT IT DOES
Streamlit Overview	Shows log-derived cluster/node assessments and incident counts
Incidents & evidence	Shows findings, correlated incidents and original log pointers
Adaptive detection	Shows relative-latency detector evidence and statistics
LangGraph AI investigation	Previews dossier; user initiates hosted/local LLM calls
cli.py	Runs deterministic analysis and exports full JSON without LLM calls
watch.py	Polls local folder for appended log bytes and rewrites reports
storage.py	Optional SQLite incident and analysis metadata summaries

How to launch on Windows

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m streamlit run app.py
```

Why was the uploaded JSON missing the AI response?

Your `solr_rca_report.json` was a deterministic export from the analysis pipeline. It contained 21,609 events, findings, two incidents and health assessments. The optional real LLM result was displayed in Streamlit and saved in `st.session_state["last_ai"]`, not automatically merged into the ordinary exported report. Your separately saved Streamlit text does contain the completed model outputs.

Periodically arriving files

`watch.py` reads appended bytes, maintains bounded per-file in-memory tails, and periodically re-runs analysis. It detects some truncation/rotation cases and writes an updated JSON report using temporary-file replacement. It is a prototype poller, not a durable streaming processor with persistent offsets, event-time watermarks or exactly-once semantics.

16. Evaluation: what passed and what it proves

`evaluate.py` regenerates eight scenario datasets at a fixed seed, analyzes them using both the older baseline and upgraded analyzer, and matches detected incidents to the injected scenario manifest by fault family, affected nodes and approximate time. It does not test whether the LLM discovered an independently verified root cause.

SCENARIO	EXPECTED	OLD BASELINE	UPGRADED
normal	0	0	0
gc	1	2	1
zk	1	1	1
slow_no_gc	1	0	1
benign_zk	0	1	0
oom	1	1	1
independent	2	3	2
mixed	2	3	2

The rerun from the V4.1 ZIP produced 8 matched injected incidents, 0 unmatched detected incidents, and 0 missed injected incidents across 8 scenarios and 82,955 generated lines. Synthetic precision/recall/F1 are all 1.0 under this matching rule.

What do the terms mean?

True positive (TP): detected incident matches an injected positive scenario. **False positive (FP):** reported incident matches no expected scenario. **False negative (FN):** expected injected incident not detected. Precision = TP/(TP+FP). Recall = TP/(TP+FN). F1 combines precision and recall.

> **Major evaluation limitation**
> These scenarios and the detector were built from related assumptions. Perfect synthetic scenario matching does not establish production detection accuracy, hallucination-free model answers, or independent causal RCA correctness. For that we need real operator-labeled incidents and held-out model comparisons.

Unit tests versus scenario evaluation

Unit tests exercise parsers, GC event classification, evidence validation, diagnostic allowlists, budgets, HTTP error handling and LangGraph routing behavior. Scenario evaluation tests end-to-end incident-family matching on generated files. The successful Groq demonstration separately proves that the two hosted LLM stages ran for the Node A synthetic incident.

17. Production architecture and known gaps

The case study explicitly allows a non-production-complete prototype. A production version would keep the normalized-event and incident/evidence contracts but replace the local I/O and state-management assumptions.

TODAY	PRODUCTION DIRECTION
Local log files / polling	Collectors with durable checkpoints and rotation handling
In-memory event lists	Incremental event-time windows with bounded state
Operator-supplied topology	Live Solr node/shard/replica/ZooKeeper state
Log-only condition labels	Combine metrics, liveness, service SLOs and cluster APIs
Rule-based/manual checks	Evidence-backed, risk-aware conditional remediation plan
Hosted LLM without redaction pipeline	Secrets management, PII/credential masking, egress policy
No durable graph checkpointing	Checkpointed, resumable human-review workflow
Generator-defined labels	Independently labeled real incidents and model ablations

Why not add a vector database or fine-tune now?

Our runbook set is tiny and relevant log events are already selected deterministically. A vector index would add infrastructure without demonstrated value. Fine-tuning would require a representative incident corpus and labels. More agents would create additional routing and safety complexity. None is necessary to defend this bounded case-study architecture.

Four explicit nonclaims

- `NO_ANOMALY_OBSERVED` does not mean the node is confirmed alive or healthy.
- `medium` confidence is a qualitative heuristic, not a measured probability.
- Model citations existing in the dossier do not prove causal truth.
- Simulated coupling of Solr and GC records does not prove true JVM timing mechanics.

18. Interview defense: questions and grounded answers

1. Why are Solr and GC logs generated separately but coupled?

They represent separate logging streams, yet share simulated per-node heap occupancy and the pressure flag. Requests change heap state, GC reclaims from it, and pressure changes both symptom families. This is useful for testing correlation but not a physical JVM simulation.

2. Why no LLM for parsing and detection?

Python can extract exact timestamps, heap values, QTime and error counts deterministically, cheaply and with source provenance. We use the model only where human-style interpretation is useful.

3. Why do you need MAD if you already have a 1000 ms slow-query rule?

An increase from 100 to 700 ms is severe relative to baseline but never crosses 1000 ms. MAD/robust Z captures unusual relative behavior and CUSUM/persistence limits isolated spikes.

4. Explain the CUSUM threshold.

It accumulates `x - median - allowance` when latency is elevated and clips at zero. We compare to a heuristic shift threshold, then require three consecutive surprising-and-shifted eligible minutes.

5. What does DSU actually do?

The domain rules create permitted pairwise links. DSU groups connected findings into incidents efficiently; it is not the algorithm that determines whether a link is causally valid.

6. How do you avoid merging unrelated incidents?

Require close first-seen timestamps, compatible anomaly families and the same node, except for specially permitted cross-node coordination events sharing declared ZooKeeper topology.

7. Why a two-stage LLM workflow?

Stage 1 identifies uncertainty and chooses one read-only tool. Python obtains targeted evidence. Stage 2 revises the hypotheses. We have demonstrated feasibility, but not a measured accuracy advantage over one stage.

8. Why LangGraph instead of plain Python?

Plain Python could work. LangGraph exposes state, conditional retries, tool transitions and fallback clearly, and offers a path toward checkpointing.

9. How do you prevent hallucinations?

Limit evidence, require real event IDs, validate JSON structure and confidence, forbid model-controlled remediation and keep fallback. However semantic support and causality are not guaranteed.

10. How does cluster health work without a live Solr API?

We deliberately label it log-derived. A multi-node critical incident produces CRITICAL, but authoritative health requires live node, shard and replica state.

11. Do you have real-world accuracy metrics?

No. We have regression tests and 8 controlled synthetic scenarios. We would need independently labeled genuine incidents and operator review for real accuracy.

12. Why not deploy Kafka now?

The goal is a same-day reproducible prototype. At large scale, durable ingestion, backpressure and event-time processing may justify a broker, but it is not required for this demonstration.

13. What if Groq fails or returns 429?

Deterministic monitoring continues, and LangGraph falls back after bounded attempts. Provider-aware 429 waiting and checkpointed resumption are appropriate production upgrades.

14. What are the biggest weaknesses?

Unverified causal semantics, synthetic scenario bias, heuristic thresholds, no live cluster-state integration, incomplete remediation actions and no production-scale ingestion.

19. Rehearsal: a five-minute live walkthrough

Step 1: Launch Streamlit

Use the bundled demo; describe synthetic sources and input scope.

Step 2: Show deterministic results

Point to 21,609 parsed events, 13 findings and 2 incidents.

Step 3: Explain the adaptive anomaly

Show products/node-a 106 ms baseline vs 696 ms p95 and robust Z 24.1.

Step 4: Explain correlation

Show Node A memory/query family linking and the separate A+B ZooKeeper group.

Step 5: Open the model dossier

Show six log excerpts, exact IDs, aggregate numbers and untrusted excerpts.

Step 6: Show the two-stage execution

Explain first hypothesis, allowlisted GC/query diagnostic, 504 examined events, final cited proposal.

Step 7: End with limitations

Say temporal co-occurrence is not proof, customer impact is unknown and live cluster state is not checked.

PowerShell commands

```
# Start dashboard
.\.venv\Scripts\python.exe -m streamlit run app.py

# Offline unit tests
.\.venv\Scripts\python.exe -m unittest discover -s tests -v

# Regenerate eight-scenario evaluation
.\.venv\Scripts\python.exe evaluate.py --output evaluation.json

# Independent deterministic JSON export
.\.venv\Scripts\python.exe cli.py --dir demo_logs --nodes node-a,node-b,node-c --output
solr_rca_report.json
```

20. Final glossary and 60-second summary

TERM	PLAIN-ENGLISH MEANING
Event	One parsed timestamped log observation
Finding	An observation that triggers a detection rule
Incident	A group of correlated findings
QTime	Logged Solr query-processing duration
p95	95th percentile of logged duration measurements
Median	Middle observation after sorting
MAD	Typical absolute deviation from the median
Robust Z	Difference from median measured in scaled-MAD units
CUSUM	Accumulated evidence of a sustained upward shift
DSU / Union-Find	Groups findings connected by approved correlation links
Dossier	Small curated packet of evidence for the LLM
LangGraph state	Shared data carried through workflow nodes
Allowlist	Only permitted diagnostic names may be executed
Ground truth	Scenario-injection manifest used for evaluation, not detection
RCA	Hypothesis about underlying causes, not automatically a proven cause

60-second version
“We generate or ingest Solr and JVM GC logs, normalize them into timestamped evidence, detect explicit faults with hard rules and persistent query shifts with MAD/CUSUM, then group related findings using node/time/family/topology rules and DSU. Every incident has a deterministic explanation. Optionally, LangGraph sends a compact dossier to Groq GPT-OSS 120B. Stage 1 chooses a safe diagnostic; Python executes it; Stage 2 reassesses. Python validates the citations and output, and an operator reviews the RCA. The demonstrated data are synthetic and the causes are hypotheses.”

Source and verification scope

Primary project: `UseReady_Solr_RCA_Two_Stage_V4_1_Groq_Fix.zip`, including `generate_logs.py`, `parser.py`, `analyzer.py`, `agent.py`, `app.py`, `evaluate.py`, and source documentation. Requirements: `AI_Engineer_Case_Study_Solr.pdf`. Observed detector metrics: uploaded `solr_rca_report.json`. Two-stage model responses and timing: saved successful Streamlit text, 26 Oct 2026. Evaluation was rerun locally from the V4.1 source for this guide.

All client-facing incident and performance numbers in this guide refer to synthetic or model-demo data unless explicitly described otherwise. Where short code excerpts are simplified for learning, the accompanying caption or explanation indicates that they are schematic.

End of guide. The best defense is to trace one original log record through parsing, detection, correlation, investigation and validation—and then explain exactly what the result does not prove.